OSLAB

Skill-12

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_COMMANDS 20
#define MAX_ARGS 50

void execute_pipeline(char *input)
{
   char *commands[MAX_COMMANDS];
   int command_count = 0;

   // Split input using |
   char *command = strtok(input, "|");

   while (command != NULL && command_count < MAX_COMMANDS)
   {
      commands[command_count++] = command;
      command = strtok(NULL, "|");
   }

   // Create pipes
   int pipes[MAX_COMMANDS - 1][2];

   for (int i = 0; i < command_count - 1; i++)
   {
      if (pipe(pipes[i]) == -1)
      {
         perror("pipe");
         return;
```

```
    }
}

// Store process IDs
pid_t pids[MAX_COMMANDS];

// Create processes
for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] == -1)
    {
        perror("fork");
        return;
    }

    if (pids[i] == 0)
    {
        /*
         * If this is NOT the first command,
         * take input from previous pipe.
         */
        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }

        /*
         * If this is NOT the last command,
         * send output to next pipe.
         */
        if (i < command_count - 1)
        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        // Close all pipe descriptors in child
        for (int j = 0; j < command_count - 1; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }
```

```
            // Convert command into arguments
            char *args[MAX_ARGS];
            int argc = 0;

            char *token = strtok(commands[i], " \t\n");

            while (token != NULL && argc < MAX_ARGS - 1)
            {
                args[argc++] = token;
                token = strtok(NULL, " \t\n");
            }

            args[argc] = NULL;

            // Execute command
            if (argc > 0)
            {
                execvp(args[0], args);
                perror("execvp");
            }

            exit(EXIT_FAILURE);
        }
    }

    // Parent closes all pipes
    for (int i = 0; i < command_count - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    // Wait for all child processes
    for (int i = 0; i < command_count; i++)
    {
        waitpid(pids[i], NULL, 0);
    }
}

int main()
{
    char input[MAX_INPUT];

    while (1)
```

```
{
    printf("myshell$ ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        continue;
    }

    // Exit shell
    if (strcmp(input, "exit") == 0)
    {
        break;
    }

    // Execute pipeline if | exists
    if (strchr(input, '|') != NULL)
    {
        execute_pipeline(input);
    }
    else
    {
        // Normal command
        char *args[MAX_ARGS];
        int argc = 0;

        char *token = strtok(input, " \t\n");

        while (token != NULL && argc < MAX_ARGS - 1)
        {
            args[argc++] = token;
            token = strtok(NULL, " \t\n");
        }

        args[argc] = NULL;

        pid_t pid = fork();
```

```
        if (pid == 0)
        {
            execvp(args[0], args);
            perror("execvp");
            exit(EXIT_FAILURE);
        }
        else
        {
            waitpid(pid, NULL, 0);
        }
    }
  }

  return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~$ cd ~/OSLab
lahari@DESKTOP-AIC33F7:~/OSLab$ nano mini_shell.c
lahari@DESKTOP-AIC33F7:~/OSLab$ gcc mini_shell.c -o mini_shell
lahari@DESKTOP-AIC33F7:~/OSLab$ ./mini_shell
myshell$ ls | wc -l
30
myshell$ ls | grep txt | wc -l
2
myshell$ cat mini_shell.c | grep include | wc -l
5
myshell$ cat mini_shell.c | grep int | sort | uniq | wc -l
9
myshell$
```